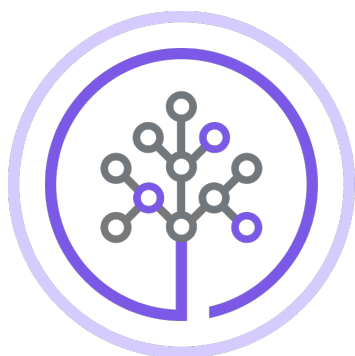




Skills
Network

Fashion-MNIST Project

Table of Contents

In this project, you will classify Fashion-MNIST dataset using convolutional neural networks.

- [Preparation](#)
- [Questions 1: Create a Dataset Class](#)
- [Define Softmax, Criterion function, Optimizer and Train the Model](#)

Estimated Time Needed: 30 min

Preparation

Download the datasets you needed for this lab.

The following are the PyTorch modules you are going to need

```
In [1]: %%time
        %pip install pandas numpy matplotlib
        %pip install torch==2.8.0+cpu torchvision==0.23.0+cpu torchaudio==
        --index-url https://download.pytorch.org/whl/cpu
```

Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-packages (3.0.1)
Requirement already satisfied: numpy in /opt/conda/lib/python3.12/site-packages (2.4.2)
Requirement already satisfied: matplotlib in /opt/conda/lib/python3.12/site-packages (3.10.8)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (24.2)
Requirement already satisfied: pillow>=8 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in /opt/conda/lib/python3.12/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
Looking in indexes: <https://download.pytorch.org/whl/cpu>
Requirement already satisfied: torch==2.8.0+cpu in /opt/conda/lib/python3.12/site-packages (2.8.0+cpu)
Requirement already satisfied: torchvision==0.23.0+cpu in /opt/conda/lib/python3.12/site-packages (0.23.0+cpu)
Requirement already satisfied: torchaudio==2.8.0+cpu in /opt/conda/lib/python3.12/site-packages (2.8.0+cpu)
Requirement already satisfied: filelock in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (4.12.2)
Requirement already satisfied: setuptools in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (75.8.0)
Requirement already satisfied: sympy>=1.13.3 in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (1.14.0)
Requirement already satisfied: networkx in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (3.6.1)
Requirement already satisfied: jinja2 in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (3.1.5)
Requirement already satisfied: fsspec in /opt/conda/lib/python3.12/site-packages (from torch==2.8.0+cpu) (2025.12.0)
Requirement already satisfied: numpy in /opt/conda/lib/python3.12/site-packages (from torchvision==0.23.0+cpu) (2.4.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /opt/conda/lib/python3.12/site-packages (from torchvision==0.23.0+cpu) (12.1.1)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /opt/conda/lib/python3.12/site-packages (from sympy>=1.13.3->torch==2.8.0+cpu) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /opt/conda/lib/python3.12/site-packages (from jinja2->torch==2.8.0+cpu) (3.0.2)
Note: you may need to restart the kernel to use updated packages.

CPU times: user 39.5 ms, sys: 17.1 ms, total: 56.6 ms
Wall time: 2.49 s

```
In [2]: # !pip install torch
        # !pip install torchvision
        # !pip install matplotlib
```

```
In [3]: # PyTorch Modules you need for this lab

        from torch.utils.data import Dataset, DataLoader

        from torchvision import transforms
        import torch
        import torch.nn as nn
        import torchvision.transforms as transforms
        import torchvision.datasets as dsets
        torch.manual_seed(0)
```

Out[3]: <torch._C.Generator at 0x710551d7a590>

Import Non-PyTorch Modules

```
In [4]: # Other non-PyTorch Modules

        from matplotlib.pyplot import imshow
        import matplotlib.pyplot as plt

        from PIL import Image
```

```
In [5]: def show_data(data_sample):
        plt.imshow(data_sample[0].numpy().reshape((IMAGE_SIZE, IMAGE_SIZE)))
        plt.title('y = ' + str(data_sample[1]))
```

Questions 1: Create a Dataset Class

In this section, you will load a Dataset object, but first you must transform the dataset. Use the `Compose` function to perform the following transforms:.

1. Use the transforms object to `Resize` to resize the image.
2. Use the transforms object to `ToTensor` to convert the image to a tensor.

You will then take a screen shot of your validation data.

Use the Compose function to compose the transforms

```
In [6]: #Hint:

IMAGE_SIZE = 16

transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
transforms.ToTensor() #
composed = transforms.Compose([transforms.Resize((IMAGE_SIZE, IMAG
```

Create two dataset objects for the Fashion MNIST dataset. One for training data called `dataset_train` and one for validation data `dataset_val`. You will be asked to take a screenshot of several samples.

Hint: `dsets.FashionMNIST(root= '.fashion/data', train=???, transform=composed, download=True)`

```
In [7]: # Create training and validation datasets
dataset_train = dsets.FashionMNIST(
    root='.fashion/data',
    train=True,
    transform=composed,
    download=True
)

dataset_val = dsets.FashionMNIST(
    root='.fashion/data',
    train=False,
    transform=composed,
    download=True
)

print(f"Training samples: {len(dataset_train)}")
print(f"Validation samples: {len(dataset_val)}")
```

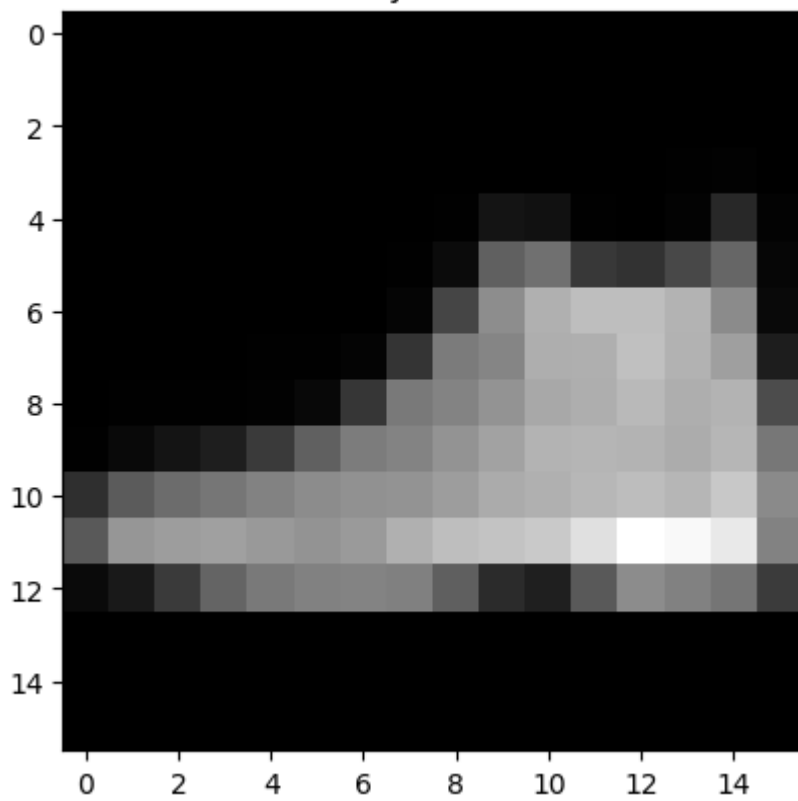
```
100%|██████████| 26.4M/26.4M [00:01<00:00, 21.7MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 356kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 6.32MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 11.8MB/s]
```

```
Training samples: 60000
Validation samples: 10000
```

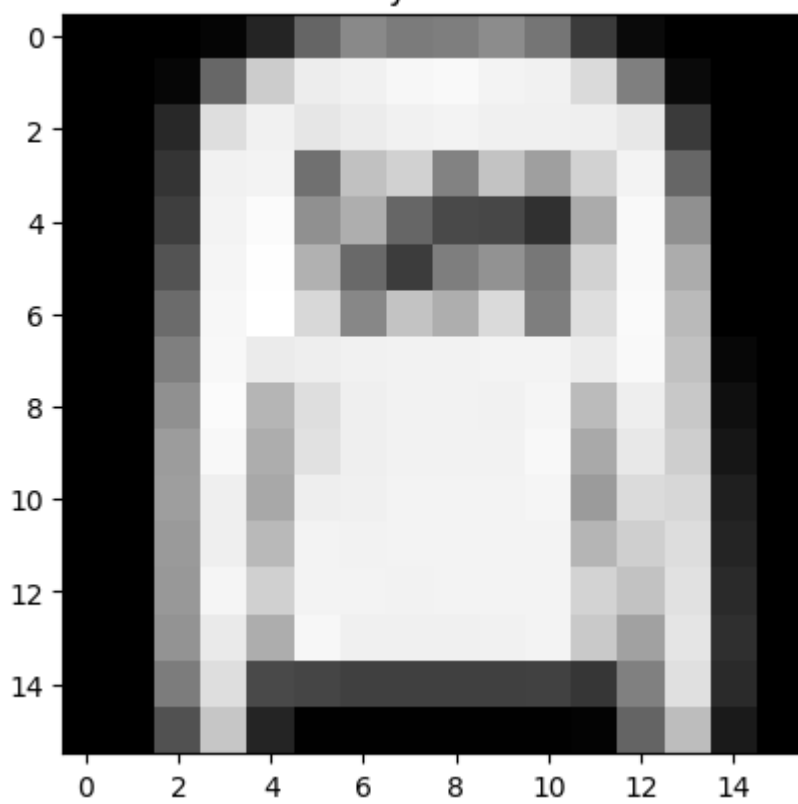
```
In [8]: for n,data_sample in enumerate(dataset_val):

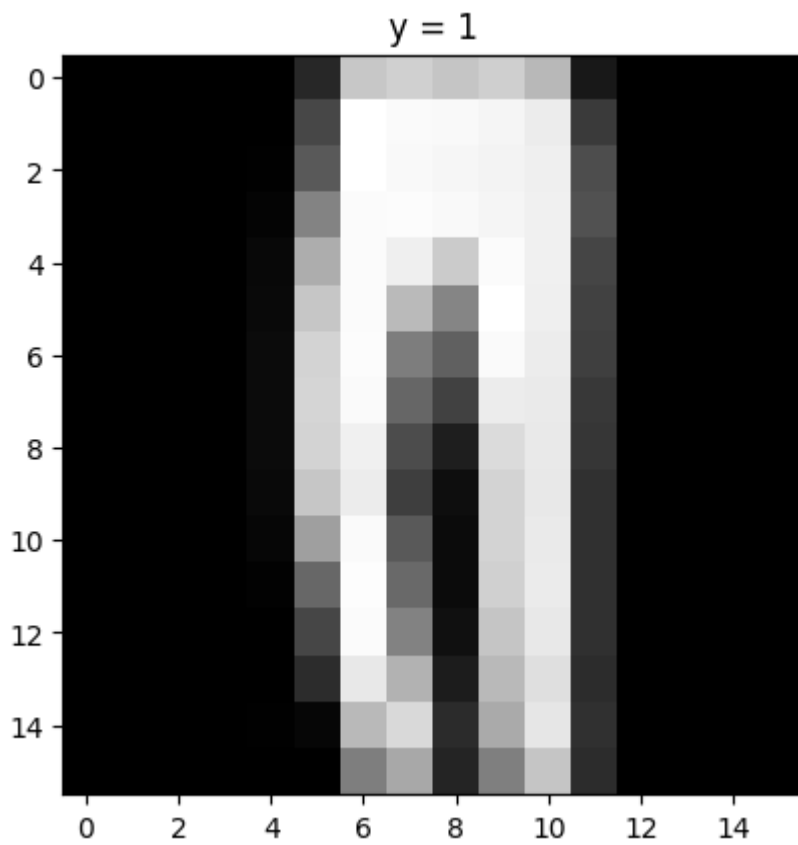
    show_data(data_sample)
    plt.show()
    if n==2:
        break
```

$y = 9$



$y = 2$





Questions 2

Create a Convolutional Neural Network class using ONE of the following constructors. Train the network using the provided code then provide a screenshot of your training cost and accuracy with your validation data.

Constructor using Batch Norm

```
In [9]: class CNN_batch(nn.Module):

    # Constructor
    def __init__(self, out_1=16, out_2=32, number_of_classes=10):
        super(CNN_batch, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=out_1, k
        self.conv1_bn = nn.BatchNorm2d(out_1)

        self.maxpool1=nn.MaxPool2d(kernel_size=2)

        self.cnn2 = nn.Conv2d(in_channels=out_1, out_channels=out_
        self.conv2_bn = nn.BatchNorm2d(out_2)

        self.maxpool2=nn.MaxPool2d(kernel_size=2)
        self.fc1 = nn.Linear(out_2 * 4 * 4, number_of_classes)
        self.bn_fc1 = nn.BatchNorm1d(10)

    # Prediction
    def forward(self, x):
        x = self.cnn1(x)
        x=self.conv1_bn(x)
        x = torch.relu(x)
        x = self.maxpool1(x)
        x = self.cnn2(x)
        x=self.conv2_bn(x)
        x = torch.relu(x)
        x = self.maxpool2(x)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        x=self.bn_fc1(x)
        return x
```

Constructor for regular Convolutional Neural Network

```
In [10]: class CNN(nn.Module):

    # Constructor
    def __init__(self, out_1=16, out_2=32, number_of_classes=10):
        super(CNN, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=1, out_channels=out_1, k
        self.maxpool1=nn.MaxPool2d(kernel_size=2)

        self.cnn2 = nn.Conv2d(in_channels=out_1, out_channels=out_
        self.maxpool2=nn.MaxPool2d(kernel_size=2)
        self.fc1 = nn.Linear(out_2 * 4 * 4, number_of_classes)

    # Prediction
    def forward(self, x):
        x = self.cnn1(x)
        x = torch.relu(x)
        x = self.maxpool1(x)
        x = self.cnn2(x)
        x = torch.relu(x)
        x = self.maxpool2(x)
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        return x
```

train loader and validation loader

```
In [11]: train_loader = torch.utils.data.DataLoader(dataset=dataset_train,
test_loader = torch.utils.data.DataLoader(dataset=dataset_val, bat
```

Convolutional Neural Network object

```
In [12]: # Instantiate the CNN with Batch Normalization
model = CNN_batch(out_1=16, out_2=32, number_of_classes=10)
print(model)
```

```
CNN_batch(
  (cnn1): Conv2d(1, 16, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2))
  (conv1_bn): BatchNorm2d(16, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1),
padding=(2, 2))
  (conv2_bn): BatchNorm2d(32, eps=1e-05, momentum=0.1,
affine=True, track_running_stats=True)
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0,
dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=512, out_features=10, bias=True)
  (bn_fc1): BatchNorm1d(10, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
)
```

Create the objects for the criterion and the optimizer named `criterion` and `optimizer`. Make the optimizer use SGD with a learning rate of 0.1 and the optimizer use Cross Entropy Loss

```
In [13]: # Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
```

Code used to train the model

```

In [14]: import time
start_time = time.time()

cost_list=[]
accuracy_list=[]
N_test=len(dataset_val)
n_epochs=5
for epoch in range(n_epochs):
    cost=0
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        z = model(x)
        loss = criterion(z, y)
        loss.backward()
        optimizer.step()
        cost+=loss.item()
    correct=0
    #perform a prediction on the validation data
    model.eval()
    for x_test, y_test in test_loader:
        z = model(x_test)
        _, yhat = torch.max(z.data, 1)
        correct += (yhat == y_test).sum().item()
    accuracy = correct / N_test
    accuracy_list.append(accuracy)
    cost_list.append(cost)

```

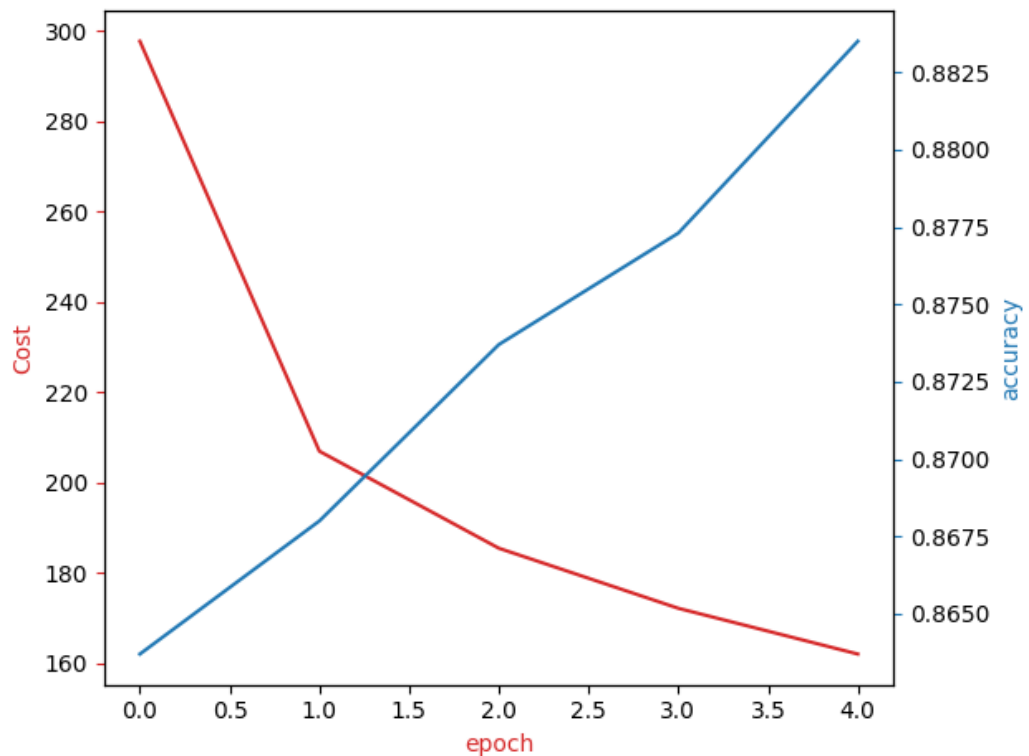
You will use the following to plot the Cost and accuracy for each epoch for the training and testing data, respectively.

```

In [15]: fig, ax1 = plt.subplots()
color = 'tab:red'
ax1.plot(cost_list, color=color)
ax1.set_xlabel('epoch', color=color)
ax1.set_ylabel('Cost', color=color)
ax1.tick_params(axis='y', color=color)

ax2 = ax1.twinx()
color = 'tab:blue'
ax2.set_ylabel('accuracy', color=color)
ax2.set_xlabel('epoch', color=color)
ax2.plot(accuracy_list, color=color)
ax2.tick_params(axis='y', color=color)
fig.tight_layout()

```



dataset: <https://github.com/zalandoresearch/fashion-mnist>

About the Authors:

[Joseph Santarcangelo](#) has a PhD in Electrical Engineering, his research focused on using machine learning, signal processing, and computer vision to determine how videos impact human cognition. Joseph has been working for IBM since he completed his PhD.

Other contributors: [Michelle Carey](#), [Mavis Zhou](#)

© IBM Corporation. All rights reserved.